Restaurant Billing System

A Python application that accepts food orders, stores them in a cart, and automatically calculates the final bill.

MENU DICTIONARY	MAIN FUNCTION	PROJECT OBJECTIVE
`menu` stores food names as keys and their prices as values. Example: Dosa = ■80, Idli = ■40, Thali = ■150 and Tea = ■20.	`main()` controls the program. It creates an empty `cart` dictionary to store ordered items and their quantities.	To automate order management, item-wise cost calculation, discount, GST and final bill generation.

Code structure

```
menu = {"Dosa": 80, "Idli": 40, "Thali": 150, "Tea": 20}

def calculate_bill(cart):
    # Calculates and displays the bill

def main():
    cart = {}
    # Accepts and stores customer orders

main()
```

Key idea: The menu stores fixed prices, while the cart stores what the customer orders. The two functions separate order-taking from bill calculation.

Customer Input and Validation

The main() function uses a loop to keep accepting orders until the customer enters “Bill”.

```
while True:
    item = input("Enter item (or bill): ").strip().title()

    if item == "Bill":
        if cart:
            calculate_bill(cart)
        else:
            print("Cart is empty!")
        break

    if item not in menu:
        print("Item not on menu. Available: Dosa, Idli, Thali, Tea")
        continue
```

CODE	EXPLANATION
`while True`	Repeats the ordering process until the loop is stopped.
`strip().title()`	Removes extra spaces and standardizes the item name, for example, “ dosa ” becomes “Dosa”.
`break`	Stops the loop when the customer enters Bill.
`continue`	Skips invalid input and asks the customer again.

Quantity validation and cart update

```
try:
    qty = int(input("Enter quantity: "))
    if qty <= 0:
        print("Enter quantity greater than 0.")
        continue
except ValueError:
    print("Enter a whole number.")
    continue

cart[item] = cart.get(item, 0) + qty
```

Explanation: try-except handles non-integer input safely. Zero and negative quantities are rejected. `cart.get(item, 0) + qty` adds the new quantity to any quantity already in the cart.

Subtotal, Discount and GST

The calculate_bill(cart) function receives the cart and calculates the cost of every ordered item.

```
def calculate_bill(cart):
    subtotal = 0
    for item, qty in cart.items():
        amount = menu[item] * qty
        subtotal += amount
        print(item, "x", qty, "| Amount:", amount)

    discount = subtotal * 0.10 if subtotal > 500 else 0
    gst = (subtotal - discount) * 0.05
    total = subtotal - discount + gst
```

STEP	FORMULA / RULE	MEANING
Item amount	Menu price × quantity	Calculates the cost of one ordered item.
Subtotal	Sum of all item amounts	Total before discount and GST.
Discount	10% if subtotal is greater than ■500; otherwise ■0	Discount is applied only when the condition is true.
GST	5% × (subtotal – discount)	GST is calculated on the discounted amount.
Grand total	Subtotal – discount + GST	Final amount payable by the customer.

Example calculation: Subtotal = ■600

SUBTOTAL	DISCOUNT (10%)	GST (5%)	GRAND TOTAL
■600	■60	■27	**■567**

The discount is deducted first. GST is then added to the discounted subtotal.

Displaying the Bill and Concepts Used

The program prints the billing summary with two decimal places for each amount.

```
print("Subtotal:", f"{subtotal:.2f}")
print("Discount:", f"{discount:.2f}")
print("GST(5%):", f"{gst:.2f}")
print("Grand Total:", f"{total:.2f}")
```

Formatting: The `:.2f` format displays exactly two digits after the decimal point, such as 567.00.

PYTHON CONCEPT	HOW IT IS USED IN THE PROJECT
Dictionary	Stores menu prices and customer cart quantities.
Functions	Separates the program into order-taking and bill-calculation tasks.
while and for loops	Repeats order input and processes every item in the cart.
if-else conditions	Checks item validity, cart status, quantity and discount eligibility.
try-except	Handles invalid quantity input without crashing.
f-strings	Formats the final bill amounts to two decimal places.

Conclusion

This project demonstrates how Python fundamentals can be combined to build a practical billing application. It manages orders, validates input, calculates discounts and GST, and generates the final bill.

Possible improvements: Display the menu automatically, add the ■ symbol to the receipt, and allow customers to place multiple orders.

THANK YOU